

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Jasim Asmaan A	Reg. No.:	192421060
Date:	17/08/2026	Duration:	60 minutes

Code of Conduct

I Jasim Asmaan A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Jasim Asmaan A

Annexure A – Git & Docker Technical Assignment

AI-Based Smart Disaster Alert and Emergency Response Coordination System

Assignment Contents

- Problem Analysis
- Project Structure Design
- Git Repository Initialization
- Git Branching Strategy
- Version Control Workflow
- Commit History Analysis
- Docker Environment Design
- Dockerfile Development
- Docker Compose Design
- Container Deployment and Testing
- Benefits of Git and Docker
- Challenges and Improvements
- Architecture and Component Interaction
- Quality Attribute Analysis
- Architectural Justification
- Final Conclusion

1. Problem Analysis

1.1 Problem Statement

Natural disasters such as floods, cyclones, earthquakes, landslides and severe storms can cause significant damage to people, infrastructure and public property. During such events, people may not receive timely and location-specific warnings, while emergency response teams may face difficulties in coordinating rescue operations and available resources.

The proposed AI-Based Smart Disaster Alert and Emergency Response Coordination System monitors disaster-related information from multiple sources and identifies potential disaster situations using predefined rules and Artificial Intelligence-based analysis. The system provides location-based alerts to citizens and supports emergency response teams in coordinating rescue operations, shelters, resources and incident information.

The system aims to reduce response delays by providing timely disaster warnings, maintaining incident information and supporting communication between citizens and emergency response authorities.

1.2 Project Objectives

2. To monitor and collect disaster-related information from different sources.
3. To identify potential disaster situations automatically.
4. To analyse disaster information using Artificial Intelligence techniques
5. To generate location-based disaster alerts for citizens.
6. To provide emergency information and safety instructions.
7. To allow citizens to report disaster incidents..
8. To provide allow emergency authorities to monitor reported incidents.
9. To support coordination of rescue teams and emergency resources.
10. To maintain disaster, alert and response history.
11. To provide a dashboard for monitoring disaster situations.
12. To improve security, reliability and maintainability through proper software architecture.
13. To use Git for version control and collaborative development.
14. To use Docker for consistent application deployment.

1.3 Functional Requirements

Requirement	Description
User Registration and Login securely log into the system.	Citizens and authorized emergency personnel can securely log into the system..
Location Management current location.	The system allows citizens to provide or update their current location.
Disaster Monitoring available sources.	The system monitors disaster-related information from available sources.
AI-Based Disaster Detection identifies potential disaster events.	The AI service analyses incoming information and identifies potential disaster
Risk Assessment level.	Each detected disaster is assigned a risk or severity level.
Alert Generation potentially affected locations.	The system generates alerts for affected or potentially affected locations.
Emergency Notifications instructions.	Citizens receive disaster warnings and emergency instructions.
Incident Reporting details.	Citizens can report disaster incidents with relevant details.
Alert Management alerts.	Authorized users can view and manage active disaster alerts.
Response Coordination and resources.	Emergency authorities can coordinate response teams and resources.
Shelter Information emergency shelters.	The system can provide information about available emergency shelters.
Dashboard information.	Displays disaster, alert, incident and response information.
Report Generation reports.	The system provides disaster and emergency response reports.

1.4 Non-Functional Requirements

- Security: Citizen, location and emergency response information must be protected.
- Performance: Disaster information and alerts should be processed within an acceptable response time.
- Scalability: The system should support increasing numbers of users, incidents and alerts.
- Reliability: The system should continue operating correctly during normal service failures.
- Availability: Disaster monitoring and alert services should remain available when required.
- Maintainability: Individual modules should be easy to modify, test and maintain.
- Usability: The application should be simple and understandable for citizens and emergency personnel.
- Portability: The application should run consistently using Docker containers.
- Accuracy: Disaster detection and alert generation should provide reliable results.
- Responsiveness: Critical disaster alerts should be delivered with minimum delay.

1.5 Expected Outcomes

The expected result is a working system that can collect disaster-related information, analyse potential disaster situations, determine the severity of incidents and generate location-based alerts for affected citizens.

The system also supports emergency response coordination by allowing authorized personnel to monitor incidents, coordinate response activities and manage emergency resources.

The use of Git provides controlled version management, while Docker provides a consistent environment for development, testing and deployment.

System Architecture Diagram

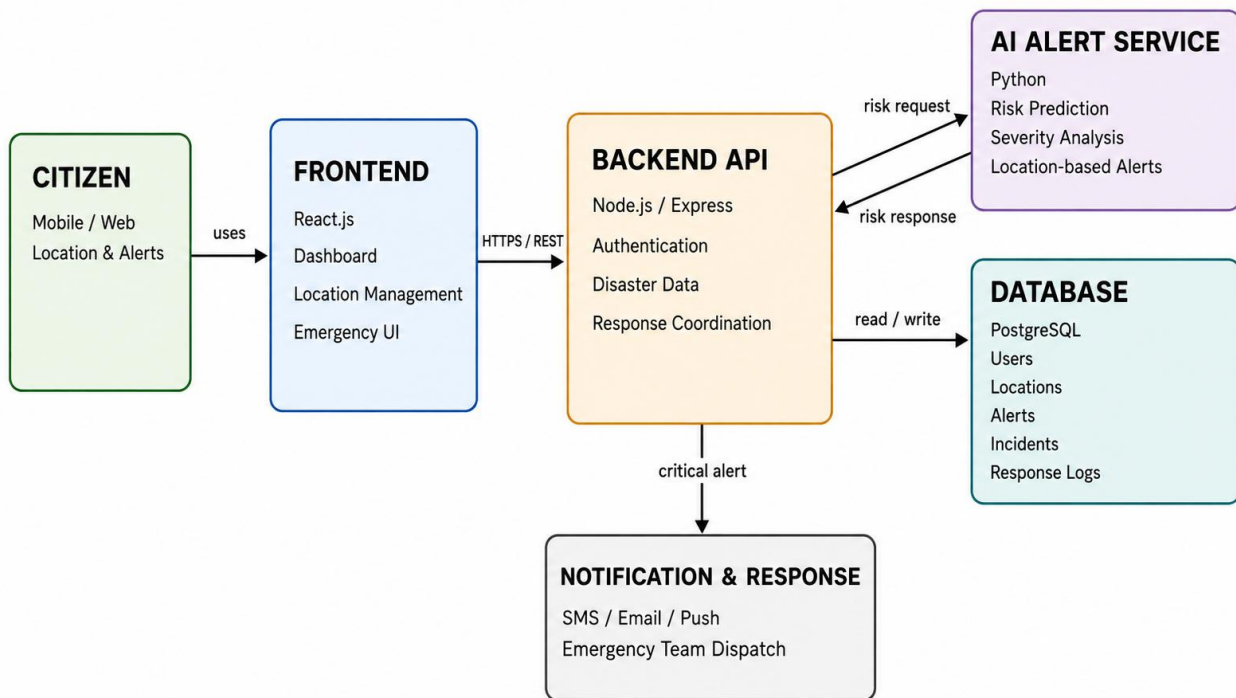


Figure 1: Overall architecture of the AI-Based Smart Disaster Alert and Emergency Response Coordination System.

2. Project Structure Design

The repository is organized into separate folders so that each component has a clear responsibility.

```
smart-disaster-alert-system/
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   └── Dockerfile
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── models/
│   │   └── middleware/
│   ├── package.json
│   └── Dockerfile
├── ai-service/
│   ├── app/
│   │   ├── main.py
│   │   ├── model.py
│   │   └── predictor.py
│   ├── requirements.txt
│   └── Dockerfile
├── database/
│   ├── init.sql
│   └── seed.sql
├── docs/
│   ├── architecture.md
│   ├── api-documentation.md
│   └── screenshots/
├── tests/
│   ├── backend/
│   └── fraud-service/
├── .gitignore
├── README.md
├── Dockerfile
└── docker-compose.yml
```

Purpose of Each Major Folder

frontend/ Contains the citizen interface, emergency dashboard and user interface.

backend/ Contains REST APIs, authentication, alert processing, incident management and communication with the database. |

ai-service/ Contains AI-based disaster detection, risk analysis and prediction logic.

database/ Contains database initialization scripts and sample data.

tests/ Contains test cases for different application components.

docs/ Contains architecture diagrams, API documentation and project screenshots.

3. Git Repository Initialization

3.1 Create the Repository

```
mkdir smart-disaster-alert-system
cd fraud-monitoring-system
```

```
git init
git branch -M main
```

```
git remote add origin https://github.com/username/ smart-disaster-alert-
system.git
```

3.2 Check Git Status

```
git status
```

This displays the files that are untracked, modified or ready to be committed.

3.3 First Commit

```
git add .
git commit -m "chore: initialize disaster alert project"
git push -u origin main
```

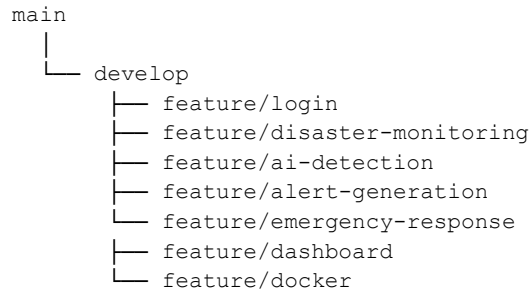
3.4 Purpose of Essential Git Files

- **.git/** – Stores Git repository information and version history.
- **.gitignore** – Prevents unwanted and sensitive files from being tracked.
- **README.md** – Explains the project and setup process.
- **main branch** – Stores stable project code.

4. Git Branching Strategy

Selected Strategy: Main + Develop + Feature Branches

A simplified Git Flow strategy is suitable for this project.



main Contains stable code suitable for release.

develop Contains integrated development code before it is released to main.

feature/* branches are used to develop individual features without directly modifying the stable branches.

```
git checkout develop
git checkout -b feature/ai-detection
```

After completing the feature:

```
git add .
git commit -m "feat: add AI disaster detection"
git push -u origin feature/ai-detection
```

Why This Strategy Is Suitable

This strategy separates stable code from active development. It reduces accidental changes to the main branch and allows different disaster management features to be developed independently. It also supports code review and easier conflict management.

5. Version Control Workflow

```
Create Issue
↓
Create Feature Branch
↓
Develop Feature
↓
Test Feature
↓
git add
↓
git commit
↓
git push
```



```
↓
Pull Request
↓
Code Review
↓
Merge to develop
↓
Testing
↓
Merge to main
```

Meaningful Commit Examples

```
git commit -m "feat: add transaction monitoring API"
git commit -m "feat: implement AI disaster risk prediction"
git commit -m "feat: add location based alert generation"
git commit -m "feat: add emergency response dashboard"
git commit -m "fix: correct disaster severity calculation"
git commit -m "test: add disaster detection test cases"
git commit -m "docs: update Docker deployment instructions"
```

Synchronization

```
git checkout develop
git pull origin develop
```

After completing the work:

```
git add .
git commit -m "feat: add emergency response coordination"
git push origin feature/emergency-response
```

Merge Conflict Resolution

```
git pull origin develop
```

Git identifies the conflicting files. The developer manually selects the correct changes, then:

```
git add .
git commit -m "fix: resolve merge conflict"
git push
```

This keeps the repository synchronized and reduces the chance of losing another developer's changes.

6. Commit History Analysis

A good commit history should clearly explain how the project developed.

```
9f31abc docs: update disaster deployment documentation
7c21d45 feat: add Docker Compose configuration
5a72c19 feat: implement AI disaster risk prediction
4b63e10 feat: add disaster monitoring API
3d52a11 feat: create incident database model
2a41b08 feat: create login functionality
1a30c07 chore: initialize project
```

Analysis

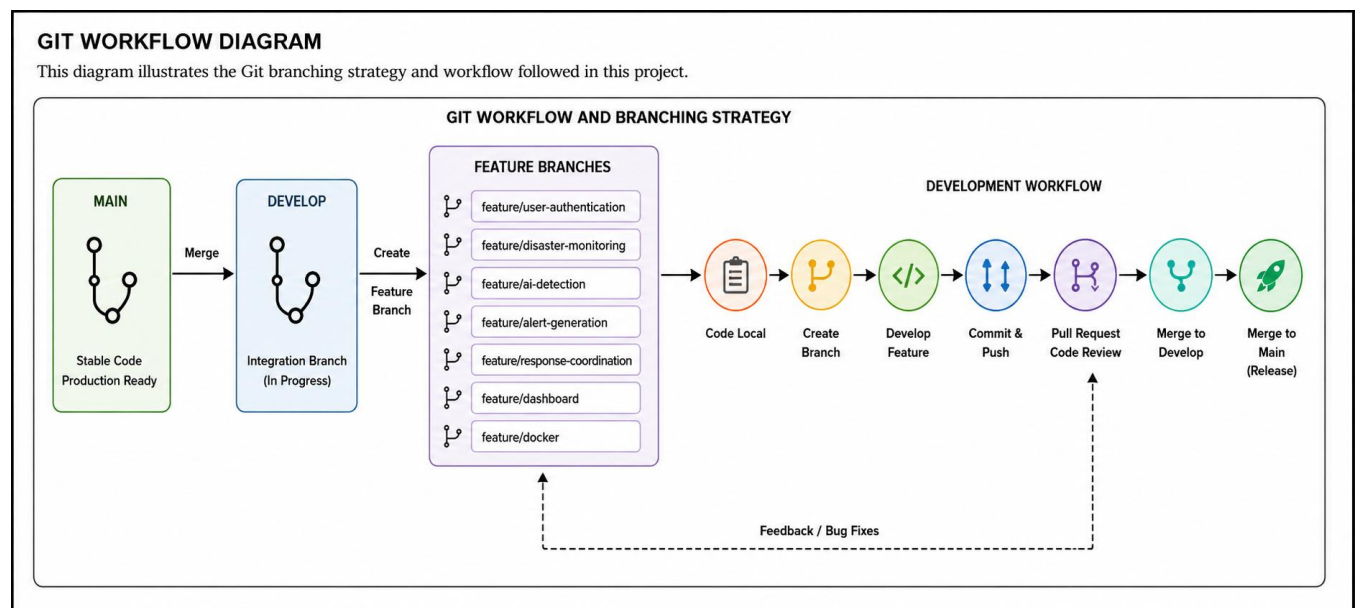
The history shows the project developing in logical stages. The first commit creates the basic repository. Authentication and incident functionality are then implemented. Disaster monitoring is added after the basic application layer. AI-based disaster detection is added after the disaster processing layer. Docker configuration is added after the application components are available.

Commit messages use a clear format such as:

```
feat:  
fix:  
test:  
docs:  
chore:
```

Benefits of Good Commit History

- Makes project development easy to track.
- Helps identify when a problem was introduced.
- Makes code review easier.
- Allows previous versions to be restored.
- Improves collaboration between developers.
- Provides evidence of systematic development.



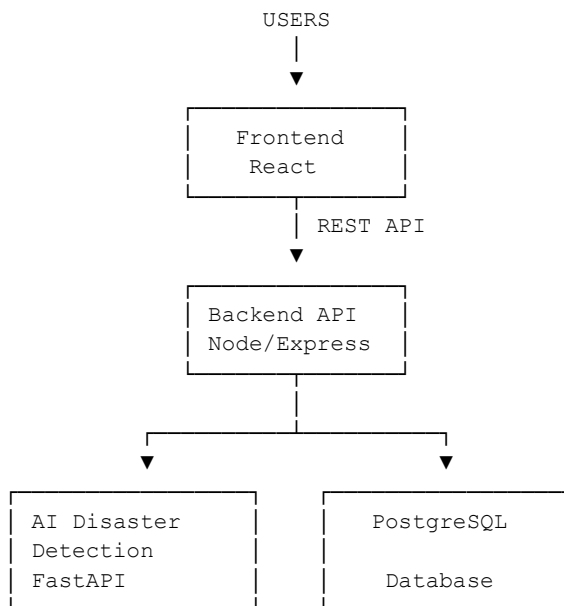
7. Docker Environment Design

Docker is suitable for this project because the system contains multiple components with different dependencies.

Components Requiring Containerization

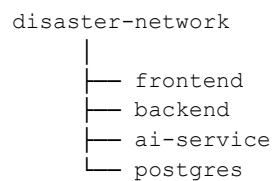
- 15. Frontend
- 16. Backend API
- 17. AI Disaster Detection Service
- 18. PostgreSQL Database

Architecture



Docker Network

All application containers communicate through a private Docker network.



The backend can communicate with PostgreSQL using the service name:

```
postgres:5432
```

8. Dockerfile Development

A Dockerfile defines how an application container is built.

Backend Dockerfile

```
FROM node:20-alpine
```

```
WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

Explanation

- FROM selects the base image.
- WORKDIR sets the application directory.
- COPY package*.json copies dependency information.
- RUN npm ci installs dependencies.
- COPY . . copies source code.
- EXPOSE documents the application port.
- CMD starts the backend application.

AI Service Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Important Dockerfile Practices

- Use small base images where appropriate.
- Do not store passwords inside the Dockerfile.
- Use environment variables for configuration.
- Use .dockerignore.
- Install only required dependencies.
- Keep images simple and maintainable.

Example .dockerignore:

```
node_modules
.git
.env
__pycache__
*.log
```

9. Docker Compose Design

Docker Compose is used because the project contains multiple services that must work together.

`docker-compose.yml`

```
services:

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - disaster-network

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgresql://frauduser:fraudpass@postgres:5432/frauddb
      AI_SERVICE_URL: http://fraud-service:8000
    depends_on:
      - postgres
      - ai-service
    networks:
      - disaster-network

  ai-service:
    build: ./ai-service
    ports:
      - "8000:8000"
    networks:
      - disaster-network

  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: disasteruser
      POSTGRES_PASSWORD: disasterpass
      POSTGRES_DB: disasterdb
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - disaster-network

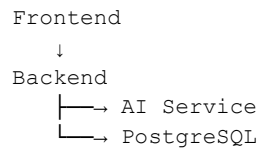
volumes:
  postgres-data:

networks:
  disaster-network:
```

Services

Service	Purpose	Port
Frontend	Citizen and emergency response user interface	3000
Backend	REST API and application processing	5000
AI Service	Disaster detection and risk analysis	8000
PostgreSQL	Data storage	5432

Dependencies



10. Container Deployment and Testing

Build the Containers

```
docker compose build
```

Start the Application

```
docker compose up -d
```

The `-d` option runs containers in detached mode.

Check Running Containers

```
docker compose ps
NAME                STATUS
frontend            running
backend              running
AI-service           running
postgres             running
```

View Logs

```
docker compose logs
docker compose logs backend
docker compose logs ai-service
```

Test the Services

Frontend: Open <http://localhost:3000>

Backend: Example endpoint: <http://localhost:5000/api/health>

AI Service: Example: <http://localhost:8000/docs>

Expected backend response:

```
{
  "status": "OK"
}
```

Test Database Connectivity

The backend sends a database request to PostgreSQL. A successful response confirms that Backend → PostgreSQL communication is working.

Stop Containers

```
docker compose down
```

Stop and Remove Volumes

```
docker compose down -v
```

The second command should be used carefully because it removes the database volume and stored database data.

Testing Procedure

1. Start all containers.
2. Check container status.
3. Open the frontend.
4. Log into the system.
5. Set or verify the user's location.
6. Submit a disaster incident report.
7. Backend receives the incident information.
8. Backend sends disaster information to the AI service.
9. AI service analyses the disaster information.
10. AI service calculates the disaster risk level.
11. Backend stores the incident and result in PostgreSQL.
12. Alert is generated for the affected location.
13. Dashboard displays the disaster and alert information.
14. Emergency response personnel review the incident.
15. Response teams and resources are coordinated.
16. Check logs for errors.
17. Stop and restart containers to verify deployment reliability.

11. Benefits of Git and Docker

Benefits of Git

Version Management – Git stores different versions of the project and allows previous versions to be restored.

Collaboration – Multiple developers can work on separate branches without directly modifying stable code.

Traceability – Commit history shows who made a change and what was changed.

Branching – Features can be developed independently and merged after testing.

Backup – A remote Git repository provides a copy of the source code and its history.

Benefits of Docker

Consistency – The same application environment can be used on different computers.

Portability – The application can be moved between development, testing and deployment environments.

Isolation – Each service runs in its own container with its required dependencies.

Scalability – Individual services can be replicated when the system requires higher capacity.

Easy Deployment – The complete application can be started using docker compose up -d.

Maintainability – Services are separated, making it easier to update or replace individual components.

12. Challenges and Improvements

12.1 Challenges Encountered

Dependency Management – Different components require different software versions and dependencies.

Solution: Docker containers are used to isolate dependencies.

Disaster Data Integration – Disaster information may come from different sources and may have different formats.

Solution: A backend service is used to validate and standardize incoming information before sending it to the AI service.

Container Communication – The frontend, backend, fraud service and database need to communicate correctly.

Solution: Docker Compose networking and service names are used.

Database Persistence – Data can be lost if the database container is removed without persistent storage.

Solution: A Docker volume is used for PostgreSQL.

Git Merge Conflicts – Changes made by different developers may affect the same files.

Solution: Feature branches, regular synchronization and code review are used.

Environment Configuration – Passwords and database connection details should not be stored directly in source code.

Solution: Environment variables and .env configuration are used.

Security & Privacy – Citizen location and emergency information may be sensitive.

Solution: Authentication, authorization, encrypted communication, input validation and secure secret management should be implemented.

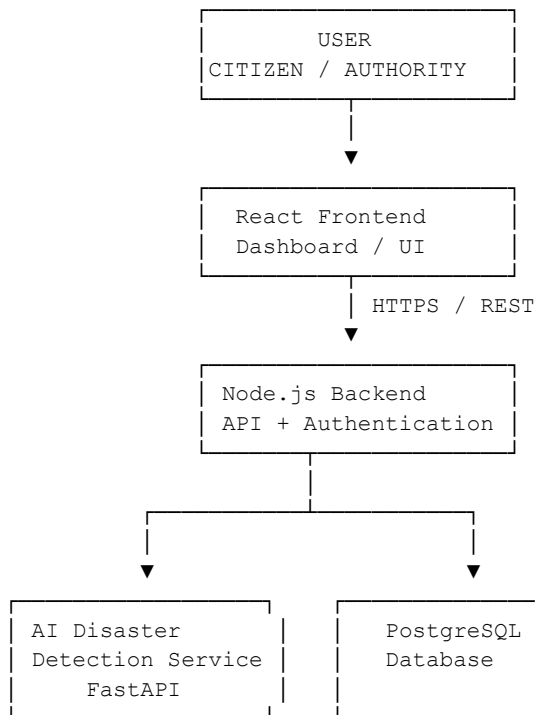
Alert Reliability – Critical disaster alerts must reach users with minimum delay.

Solution: The alert generation service should be monitored and designed to handle temporary communication failures.

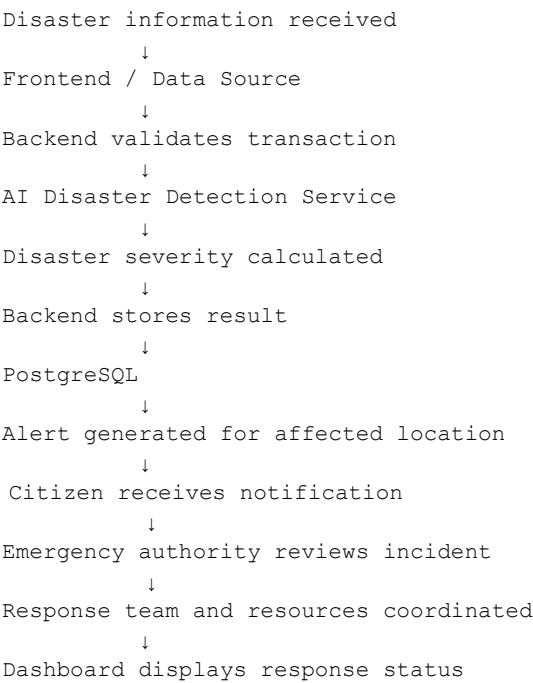
Future Improvements

1. Add advanced machine-learning models for improved disaster prediction accuracy.
 2. Add real-time integration with weather and disaster information sources.
 3. Add SMS, email and mobile push notifications for critical alerts.
 4. Add role-based access control for emergency authorities.
 5. Add automated GitHub Actions CI/CD.
 6. Add automated unit and integration testing.
 7. Add centralized logging and monitoring.
 8. Use Docker secrets or a secure secret-management system.
 9. Add database backup and recovery mechanisms.
 10. Deploy the containers to a cloud platform.
 11. Add load balancing for large numbers of users during disasters.
 12. Add an audit trail for important emergency response operations.
 13. Add real-time emergency response team tracking.
 14. Add multilingual disaster alerts.
 15. Add offline or low-connectivity support for critical emergency information.
- Architecture and Component Interaction

Overall Architecture



Disaster Workflow



Component Interaction

Component	Interacts With	Purpose
Frontend	Backend	Sends requests and displays alerts /results
Backend	PostgreSQL	Stores and retrieves disaster and user information.
Backend	AI Service	Requests disaster analysis and risk assessment.
AI Service	Backend	Returns disaster classification and risk result.
Backend	Notification Servic	Sends alerts to affected users.
Backend	Emergency Dashboard	Provides incident and response information.
PostgreSQL	Backend	Provides persistent storage
Docker Compose	All containers	Manages services and networking

Quality Attribute Analysis

Security – Authentication and authorization protect system access. Sensitive configuration values should be stored as environment variables or secure secrets. Citizen location and emergency information should be protected from unauthorized access.

Scalability – The frontend, backend and AI disaster detection services are separated into containers, allowing individual services to be scaled independently.

Performance – The backend handles API requests while the AI service performs disaster analysis separately. This separation prevents all processing from being placed in one component.

Reliability – Database persistence through Docker volumes helps protect stored transaction data. Service health checks and proper error handling can further improve reliability.

Maintainability – The system is divided into frontend, backend, fraud detection and database components. A change in one component can therefore be made without rewriting the complete system.

Availability – Containerized services can be restarted independently. In a production environment, multiple instances and health monitoring can further improve availability.

Usability – The system provides a simple interface for citizens to receive alerts and report incidents, while emergency personnel receive a dedicated monitoring dashboard.

Accuracy – AI-based disaster analysis and predefined rules can be combined to improve the identification and severity assessment of disaster situations.

Architectural Justification

Why Layered/Service-Based Architecture?

The selected architecture separates the system into clear responsibilities:

```
Presentation Layer
    ↓
Application/API Layer
    ↓
AI Disaster Service
    ↓
Data Layer
```

This is suitable because the project has different responsibilities such as user interaction, transaction processing, fraud analysis and data storage.

Advantages

Modularity – Each major component performs a specific responsibility.

Scalability – Individual services can be scaled when the workload increases.

Maintainability – Developers can modify one component without affecting the complete system.

Security – Security controls can be applied at the API and service levels.

Reliability – Failure in one service can be isolated and handled without necessarily stopping the entire system.

Portability – Docker allows the application to run consistently across different environments.

Fast Response – Separating AI analysis and alert processing from the main frontend improves the ability to process emergency information efficiently.

Extensibility – Additional disaster types, notification channels and emergency response services can be added without redesigning the complete system.

Final Conclusion

The proposed ****AI-Based Smart Disaster Alert and Emergency Response Coordination System**** provides a structured solution for monitoring disaster-related information, identifying potential disaster situations and delivering timely alerts to affected citizens.

The system also supports emergency response coordination by maintaining incident information and allowing authorized personnel to monitor disasters, coordinate response teams and manage emergency resources.

Git is used for version control, branching, collaboration, commit tracking and source-code management. Docker is used to containerize the frontend, backend, AI disaster detection service and database, providing consistency and portability.

The architecture separates presentation, application processing, AI-based disaster analysis, alert generation, emergency response coordination and data storage. This improves security, scalability, performance, reliability and maintainability.

The combination of Git, Docker, Docker Compose and a modular software architecture provides a suitable foundation for developing, testing and deploying the proposed AI-based disaster management application.

DOCKER COMPOSE ARCHITECTURE

